KU LEUVEN

FACULTEIT INGENIEURSWETENSCHAPPEN

Race conditions
Lab 2
Geavanceerde computerarchitectuur

Wouter Fouquet
Florian Vandenbogaerde
Master elektronica ICT
Academiejaar 2024-2025

CONTENTS

Exercise **1**

1 Methodology **2**
1.1 Objective 2
1.2 Results 3
1.3 Guiding questions 3

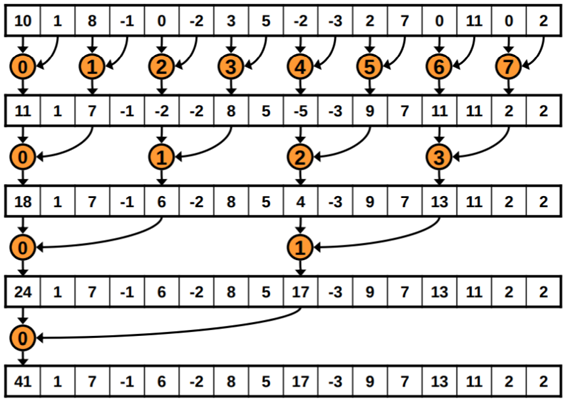
Conslusion **5**

EXERCISE

- Implement max detection using a nested for loop on the CPU, a GPU kernel employing automic operation and a GPU kernel using reduction.
- Use a 1D input array of your liking. Assess the dependency of the execution time on the input array's length. Timing examples are provided in the cheat sheet.

1.1 Objective

This lab session was on race conditions and how that we can avoid them. In this assignment, race conditions can arise when multiple threads access the array elements simultaneously, while the threads from a previous step are still running. This is illustrated on figure 1.1. If one of the threads on the first row is not finished and a thread on the second row starts executing we would get wrong results. This is the reason why we need synchronization.



Figuur 1.1: Array reduction

In this report, we explore the performance of CPU and GPU implementations for finding the maximum value in an array. The primary focus is on comparing traditional CPU algorithms with GPU-accelerated methods, specifically utilizing atomic operations and reduction techniques. The CPU implementation employs a nested loop approach, iterating through the array to determine the maximum value. Conversely, the GPU implementations leverage parallel processing capabilities, allowing multiple threads to operate simultaneously on the data. The first GPU kernel utilizes atomic operations to safely update the maximum value across threads, while the second GPU kernel employs a shared memory reduction strategy to efficiently compute the maximum. Performance metrics, including execution time, are measured for both CPU and GPU approaches, providing insights into the advantages of parallel computation in handling large datasets.

1.2 Results

The first test we conducted utilized a nested for-loop on the CPU. We then implemented a GPU kernel employing atomic operations. Finally, we tested a GPU kernel using reduction. This last approach follows the same principle as shown in Figure 1.1, but instead of summing two values, it compares them and retains the maximum. Table 1.1 presents an overview of the timing results obtained during this lab session.

Array Size	Threads per Block	Number of Blocks	CPU: Nested for loop [µs]	GPU: atomic operation [µs]	CPU: reduction [µs]
8	8	1	0,3	101,37	27,64
1024	1024	1	3077,5	101,79	31,2
2048	1024	2	7083,5	154,91	44,03
10240	1024	10	176225	122,88	31,07

Tabel 1.1: Vergelijking van uitvoeringstijden voor array-flipping op GPU en CPU

1.3 Guiding questions

Question 1: Is the result correct?

Answer 1: For all three tests, we use an array initialized with predefined values. For simplicity, each element's value is set equal to its index. This ensures that the maximum value is always equal to the size of the array. Regardless of the algorithm used, we consistently obtain the correct result.

Question 2: How does time increase as a function of input size?

Answer 2: For small input sizes, the CPU outperforms the GPU due to kernel launch overhead and data transfer time. However, as input size increases, the GPU becomes significantly faster because of its parallel processing capabilities. The CPU approach scales linearly (O(N) complexity), while the reduction-based GPU approach can achieve O(log N) complexity. Atomic operations, however, may suffer from contention and do not scale as well as reduction.

Question 3: What happens if the input is larger than 2048 (how would you ensure correct calculation and are there additional synchronization steps)?

Answer 3: When handling large input sizes that exceed the number of available threads per block, the array must be processed in multiple blocks. Each block computes a local maximum using shared memory and synchronization barriers. Afterward, a second kernel or an atomic operation can be used to compute the global maximum across all block results. Efficient memory access patterns and minimizing synchronization overhead are crucial for performance in this case.

Question 4: Consider calling the kernel with much less threads than there are data elements. For example, one tenth. How would you ensure all elements get processed?

Answer 4: If the number of threads is smaller than the number of elements, each thread must process multiple elements. This can be achieved using a grid-stride loop, where each thread iterates over its assigned portion of the array using a step size equal to gridDim.x * blockDim.x. This ensures that all elements are processed even with fewer threads.

Question 5: How would you tackle a 2D array?

Answer 5: For a 2D array, we first compute the maximum value in each column and store these values in a separate 1D array. Once this intermediate array is created, we apply the same reduction method used for 1D arrays to determine the overall maximum value.

CONCLUSION

The results show that the CPU approach is inefficient for larger datasets due to its quadratic time complexity. The GPU implementation using atomic operations is significantly faster but introduces performance drawbacks due to synchronization overhead. The reduction approach on the GPU achieves the best performance by making more efficient use of shared memory and parallel processing, thereby minimizing the impact of synchronization.

From this, we can conclude that GPU-based techniques, especially reduction methods, provide a significant speedup for large datasets and operations that can be processed in parallel.

```
1
2  #include <iostream>
3  #include <chrono>
4  #include <cuda_runtime.h>
5
6  using namespace std;
7  using namespace std::chrono;
8
9  int findMaxCPU(int array[], int size) {
10     int maxVal = array[0];
11     for (int i = 0; i < size; i++) {
12         if (array[i] > maxVal) {
13             maxVal = array[i];
14         }
15     }
16     return maxVal;
17 }
18
19 __global__ void findMaxGPUAtomic(int *array, int *maxVal, int size) {
20     int index = threadIdx.x + blockIdx.x * blockDim.x;
21     if (index < size) {
22         atomicMax(maxVal, array[index]);
23     }
24 }
25
26 __global__ void findMaxGPUReduction(int *array, int *maxVal, int size) {
27     extern __shared__ int sharedMax[];
28     int index = threadIdx.x + blockIdx.x * blockDim.x;
29
30     //unique ID for each thread in the block
31     int tid = threadIdx.x;
32
33     // copy the value of the array to sharedMax
34     if (index < size) {
35         sharedMax[threadIdx.x] = array[index];
36     } else {
37         sharedMax[threadIdx.x] = INT_MIN;
38     }
39     __syncthreads();
40
41     // Reduce in pairs
42     for (int stride = blockDim.x / 2; stride > 0; stride /= 2) {
43         if (threadIdx.x < stride) {
44             sharedMax[threadIdx.x] = max(sharedMax[threadIdx.x], sharedMax[
                   threadIdx.x + stride]);
45         }
46         __syncthreads();
47     }
48
49     // De thread met tid == 0 slaat de maximale waarde op in maxVal
50     if (threadIdx.x == 0) {
51         atomicMax(maxVal, sharedMax[0]);
52     }
53
54 }
55
56
57
58 int main() {
59     //create array
```

```
60      const int sizeArray = 10240;
61      int data[sizeArray];
62      for (int i = 0; i < sizeArray; i++) data[i] = i;
63
64      int NOThreads = 1024;
65      int NOBlocks = 10;
66
67      //NESTED FOR LOOP ON CPU
68
69      //Start Timer
70      auto startCpu = high_resolution_clock::now();
71
72      int maxVal = findMaxCPU(data, sizeArray);
73
74      //Stop Timer
75      auto stopCpu = high_resolution_clock::now();
76      auto durationCpu = duration_cast<nanoseconds>(stopCpu - startCpu);
77
78      cout << "Maximum value CPU: " << maxVal << endl;
79
80      printf("CPU-time: %lld ns\n", durationCpu.count());
81
82
83      //GPU KERNEL EMPLOYING ATOMIC OPERATIONS
84
85      int *dataGPU, *maxValGPU;
86      int maxValGPUHost = INT_MIN;
87
88      cudaMalloc(&dataGPU, sizeArray * sizeof(int));
89      cudaMalloc(&maxValGPU, sizeof(int));
90
91      cudaMemcpy(dataGPU, data, sizeArray * sizeof(int),
            cudaMemcpyHostToDevice);
92      cudaMemcpy(maxValGPU, &maxValGPUHost, sizeof(int),
            cudaMemcpyHostToDevice);
93
94      // Start Timing GPU
95      cudaEvent_t start_cuda, stop_cuda;
96      cudaEventCreate(&start_cuda);
97      cudaEventCreate(&stop_cuda);
98      cudaEventRecord(start_cuda);
99
100     findMaxGPUAtomic<<<NOBlocks, NOThreads>>>(dataGPU, maxValGPU, sizeArray
            );
101
102     // Stop Timing GPU
103     cudaDeviceSynchronize();
104     cudaEventRecord(stop_cuda);
105     cudaEventSynchronize(stop_cuda);
106     float msCuda;
107     cudaEventElapsedTime(&msCuda, start_cuda, stop_cuda);
108
109     cudaMemcpy(&maxValGPUHost, maxValGPU, sizeof(int),
            cudaMemcpyDeviceToHost);
110
111     cout << "Maximum value GPU with atomic operation: " << maxValGPUHost <<
             endl;
112
113     printf("GPU-time with atomic operation: %f ms\n", msCuda);
114
115     cudaFree(dataGPU);
```

```
116     cudaFree(maxValGPU);
117
118
119
120
121     //GPU KERNEL USING REDUCTION
122
123     int *dataGPU2, *maxValGPU2;
124     //reset maxVal
125     int maxValGPUHost2 = INT_MIN;
126
127     cudaMalloc(&dataGPU2, sizeArray * sizeof(int));
128     cudaMalloc(&maxValGPU2, sizeof(int));
129
130     cudaMemcpy(dataGPU2, data, sizeArray * sizeof(int),
            cudaMemcpyHostToDevice);
131     cudaMemcpy(maxValGPU2, &maxValGPUHost2, sizeof(int),
            cudaMemcpyHostToDevice);
132
133
134     // Start Timing GPU
135     cudaEvent_t start_cuda2, stop_cuda2;
136     cudaEventCreate(&start_cuda2);
137     cudaEventCreate(&stop_cuda2);
138     cudaEventRecord(start_cuda2);
139
140     findMaxGPUReduction<<<NOBlocks, NOThreads, NOThreads * sizeof(int)>>>(
            dataGPU2, maxValGPU2, sizeArray);
141
142     //Stop Timing GPU
143     cudaDeviceSynchronize();
144     cudaEventRecord(stop_cuda2);
145     cudaEventSynchronize(stop_cuda2);
146     float msCuda2;
147     cudaEventElapsedTime(&msCuda2, start_cuda2, stop_cuda2);
148
149     cudaMemcpy(&maxValGPUHost2, maxValGPU2, sizeof(int),
            cudaMemcpyDeviceToHost);
150
151     cout << "Maximum value GPU with reduction: " << maxValGPUHost2 << endl;
152
153     printf("GPU-time with reduction: %f ms\n", msCuda2);
154
155
156     cudaFree(dataGPU2);
157     cudaFree(maxValGPU2);
158
159
160
161     return 0;
162 }
```